



Session & General Commands

Linux / Unix command-line fundamentals

```
$ echo "Welcome to the shell"
```

`PATH · man · echo / printf · system info · script`

What We'll Cover

01

PATH variable

How the shell locates executable programs

02

man — manual pages

Built-in reference documentation for commands

03

echo and printf

Displaying text and formatted output

04

System & session info

passwd, uname, who, w, date, stty

05

script

Recording an entire terminal session

PATH Variable

An environment variable listing directories the shell searches for executables

- A colon-separated list of directories, searched left to right
- When you type a command, the shell scans each PATH directory until it finds a matching executable
- "command not found" usually means the program isn't in any PATH directory
- Add a directory permanently via your shell config (~/.bashrc, ~/.zshrc)

```
$ echo $PATH  
/usr/local/bin:/usr/bin:/bin:/usr/sbin
```

```
$ export PATH=$PATH:/opt/tools/bin
```

```
$ which python3
```

Tip: use full paths (e.g. "add" export lines) so PATH persists across new terminal sessions.

man — Manual Pages

The built-in reference manual for commands, system calls, and config files

```
$ man ls
```

- Opens a paginated manual page in a pager (usually less)
- Navigate: Space / b to page down/up, / to search, q to quit
- `man -k <anykeyword>` — search page descriptions
- `man 5 <anycommand>` — pick a specific section (1=commands, 5=file formats, 8=admin)

Manual Sections

- | | |
|---|----------------------------|
| 1 | User commands |
| 2 | System calls |
| 3 | Library functions |
| 4 | Special files (/dev) |
| 5 | File formats & conventions |
| 6 | Games |
| 7 | Miscellaneous |
| 8 | System admin commands |

echo and printf

Two ways to write output to the terminal

echo

Prints text followed by a newline. Simple and fast.

```
$ echo "Hello, World!"
```

```
$ echo -n "no newline"
```

```
$ echo -e "Tab:\tNewline:\n!"
```

-n suppress trailing newline **-e** enable backslash escapes

printf

Formats output using a format string and conversion specifiers, like C's printf.

```
$ printf "Name: %s\n" "Sam"
```

```
$ printf "%d + %d = %d\n" 2 3 5
```

```
$ printf "%.2f\n" 3.14159
```

%s string **%d** integer **%f** float

System & Session Commands

`passwd` · `uname` · `who` · `w` · `date` · `stty`

passwd

Change a user's login password

```
$ passwd
```

uname

Print system/kernel information

```
$ uname -a
```

whoami / who

Show who is currently logged in

```
$ whoami / who
```

w

Show logged-in users and what they're running

```
$ w
```

date

Display or set the system date and time

```
$ date +%Y-%m-%d
```

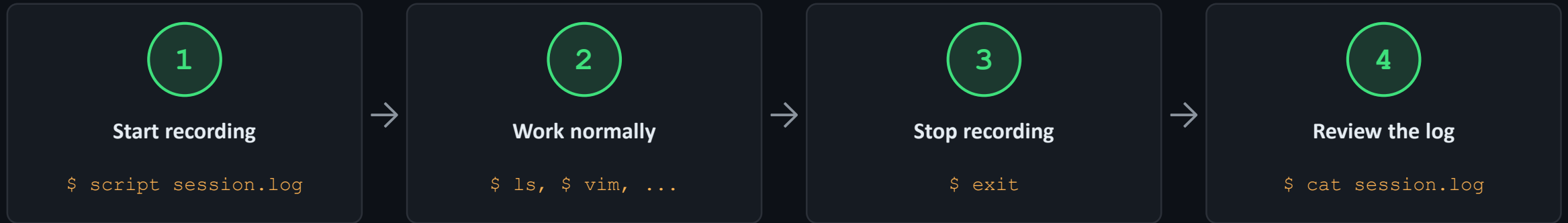
stty

View or change terminal line settings

```
$ stty -a
```

script — Recording a Terminal Session

Captures everything typed and displayed in a shell session to a log file



- Useful for creating a transcript of commands and their output — training material, bug reports, audits
- `script -a session.log` appends instead of overwriting the log file
- The captured file includes terminal control characters, so it looks best when replayed (e.g. `scriptreplay`) or viewed in a terminal

Key Takeaways



`PATH`

tells the shell where to find programs



`man`

is the first place to look for command help



`echo / printf`

write text and formatted output to the terminal



`passwd, uname, who, w, date, stty`

report and control account, system, and terminal state



`script`

records a full session transcript for later review